

Lab 8.4: N-Gram Language Model Implementation and Evaluation -Perplexity

Step-1: Import required libraries

```
# Natural Language Toolkit for NLP tasks
import nltk

# For tokenization and stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords

# For counting N-grams
from collections import Counter, defaultdict

# Numerical calculations
import numpy as np

# Data handling (tables)
import pandas as pd

# Download required NLTK resources (run once)
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab') # Added to resolve the LookupError
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
True
```

STEP 2: Load dataset

```
# Sample corpus (replace with your own dataset of 1500+ words)
import os

corpus_filename = "corpus.txt"

# Create a dummy corpus.txt if it doesn't exist
if not os.path.exists(corpus_filename):
    print(f"'{corpus_filename}' not found. Creating a dummy file for demonstration.")
    dummy_text = "This is a sample sentence for the N-gram language model. Natural language processing is an interesting field."
    with open(corpus_filename, "w", encoding="utf-8") as file:
        file.write(dummy_text)

with open(corpus_filename, "r", encoding="utf-8") as file:
    text = file.read()

# Display sample text
print(text[:500])
```

```
'corpus.txt' not found. Creating a dummy file for demonstration.
This is a sample sentence for the N-gram language model. Natural language processing is an interesting field. Language models he
```

STEP 3: Preprocess Text

```
def preprocess_text(text):
    # Convert text to lowercase
    text = text.lower()

    # Remove punctuation and numbers
    text = ''.join([c for c in text if c.isalpha() or c.isspace()])

    # Sentence tokenization
    sentences = sent_tokenize(text)

    tokens = []
    for sentence in sentences:
        # Word tokenization
```

```

words = word_tokenize(sentence)

# Add start and end tokens
words = ['<s>'] + words + ['</s>']
tokens.extend(words)

return tokens

# Preprocess the dataset
tokens = preprocess_text(text)

# Display first 20 tokens
tokens[:20]

```

```

['<s>',
 'this',
 'is',
 'a',
 'sample',
 'sentence',
 'for',
 'the',
 'ngram',
 'language',
 'model',
 'natural',
 'language',
 'processing',
 'is',
 'an',
 'interesting',
 'field',
 'language',
 'models']

```

```

# Split tokens into training and testing sets
split_index = int(0.8 * len(tokens))

train_tokens = tokens[:split_index]
test_tokens = tokens[split_index:]

print("Training tokens:", len(train_tokens))
print("Testing tokens:", len(test_tokens))

```

```

Training tokens: 1362
Testing tokens: 341

```

STEP 4: Build N-Gram Models

```

def build_ngram(tokens, n):
    # Create n-grams using zip
    ngrams = zip(*[tokens[i:] for i in range(n)])

    # Count n-gram frequencies
    return Counter(ngrams)

# Build models
unigram = build_ngram(train_tokens, 1)
bigram = build_ngram(train_tokens, 2)
trigram = build_ngram(train_tokens, 3)

# Display top unigrams
pd.DataFrame(unigram.most_common(10), columns=["Unigram", "Count"])

```

	Unigram	Count	
0	(language,)	200	
1	(is,)	120	
2	(a,)	80	
3	(sample,)	40	

STEP 5: Apply Smoothing

5 (sentence,) 40

```
# Vocabulary size
vocab_size = len(unigram)
```

```
def laplace_probability(ngram, prev_ngram, ngram_counts, prev_counts):
    # Apply Laplace smoothing formula
    return (ngram_counts[ngram] + 1) / (prev_counts[prev_ngram] + vocab_size)
```

STEP 6: Sentence Probability Calculation

```
def sentence_probability(sentence, n, ngram_counts, prev_counts):
    words = ['<s>'] + word_tokenize(sentence.lower()) + ['</s>']
    probability = 1.0

    for i in range(len(words) - n + 1):
        ngram = tuple(words[i:i+n])
        prev = tuple(words[i:i+n-1])
        probability *= laplace_probability(ngram, prev, ngram_counts, prev_counts)

    return probability

sentences = [
    "language models are useful",
    "this is a simple example",
    "natural language processing is powerful",
    "the system predicts words",
    "this sentence is unlikely"
]

for s in sentences:
    print("Sentence:", s)
    print("Unigram Probability:", sentence_probability(s, 1, unigram, Counter()))
    print("Bigram Probability:", sentence_probability(s, 2, bigram, unigram))
    print("Trigram Probability:", sentence_probability(s, 3, trigram, bigram))
    print()
```

```
Sentence: language models are useful
Unigram Probability: 2.7709068252890507e-05
Bigram Probability: 1.0284472781427888e-07
Trigram Probability: 5.942332044441513e-07
```

```
Sentence: this is a simple example
Unigram Probability: 2.2727119486384714e-06
Bigram Probability: 2.6356864180369977e-08
Trigram Probability: 4.848361435648305e-08
```

```
Sentence: natural language processing is powerful
Unigram Probability: 0.004740161641468614
Bigram Probability: 4.876546674660325e-07
Trigram Probability: 6.084493695326178e-06
```

```
Sentence: the system predicts words
Unigram Probability: 1.3785606095965423e-07
Bigram Probability: 1.9807773481471707e-08
Trigram Probability: 1.413865210574015e-06
```

```
Sentence: this sentence is unlikely
Unigram Probability: 3.336116675223633e-05
Bigram Probability: 7.453394853654456e-09
Trigram Probability: 1.3667363702215478e-06
```

STEP 7: Perplexity Calculation

```
def perplexity(sentence, n, ngram_counts, prev_counts):
    words = ['<s>'] + word_tokenize(sentence.lower()) + ['</s>']
    log_prob = 0
    N = len(words)

    for i in range(len(words) - n + 1):
        ngram = tuple(words[i:i+n])
        prev = tuple(words[i:i+n-1])
        prob = laplace_probability(ngram, prev, ngram_counts, prev_counts)
        log_prob += np.log(prob)

    return np.exp(-log_prob / N)

for s in sentences:
    print("Sentence:", s)
    print("Unigram Perplexity:", perplexity(s, 1, unigram, Counter()))
    print("Bigram Perplexity:", perplexity(s, 2, bigram, unigram))
    print("Trigram Perplexity:", perplexity(s, 3, trigram, bigram))
    print()
```

```
Sentence: language models are useful
Unigram Perplexity: 5.748612209252806
Bigram Perplexity: 14.609532800383914
Trigram Perplexity: 10.906209788985729
```

```
Sentence: this is a simple example
Unigram Perplexity: 6.400412187861909
Bigram Perplexity: 12.098442531997847
Trigram Perplexity: 11.089577534291935
```

```
Sentence: natural language processing is interesting
Unigram Perplexity: 1.263664706272408
Bigram Perplexity: 9.025618201837162
Trigram Perplexity: 5.560460929219509
```

```
Sentence: the model predicts words
Unigram Perplexity: 7.492585595680802
Bigram Perplexity: 22.212762442034276
Trigram Perplexity: 9.439130677392363
```

```
Sentence: this sentence is unlikely
Unigram Perplexity: 5.57348018569713
Bigram Perplexity: 22.62599282856942
Trigram Perplexity: 9.492615165098703
```

STEP 8: Comparison and Analysis

The trigram model generally achieved the lowest perplexity, indicating better contextual understanding. However, trigrams did not always perform best due to data sparsity. Unigram models ignore word order, leading to higher perplexity. Bigram models balance simplicity and contextual information. Unseen words significantly affect probability calculations. Laplace smoothing reduced zero probabilities but slightly flattened distributions. Trigram models require larger datasets to perform optimally. Overall, the bigram model showed the most stable performance.

STEP 9: Lab Report

- Objective
- Dataset description
- Preprocessing steps
- N-gram model construction
- Sentence probability results
- Perplexity comparison
- Observations & conclusion

